Project Requirement Sheet
Project Title: Perfumed Elegance – Perfume & Decant E-Commerce Backend API

Framework: NestJS + TypeScript

Database: PostgreSQL + TypeORM

1. Project Overview
Perfumed Elegance is a backend RESTful API for a perfume and fragrance decant e-commerce platform. The system will allow customers to explore perfumes, view details, and place orders while administrators manage the product catalog, inventory, and order processing.

Main Focus Areas:

• User management (Customer and Admin roles)

• Perfume & decant product catalog management

• Product brand and category organization

• Customer order placement and tracking

• Inventory / stock management

• Email notifications for important order actions

No frontend is required — the API will be tested using Postman and Swagger UI.

2. Core Functional Requirements (Must-have Endpoints)

#	Feature	HTTP Method	Endpoint Example	Description / Expected Behavior	Priority
1	User Registration & Login	POST	/auth/register , /auth/login	Customer registers account and logs in using email/password. Login returns JWT.	Must
2	Get Current User Profile	GET	/auth/me	Protected route returning authenticated user information.	Must

3	Create Brand	POST	/brands	Admin creates perfume brands (e.g., Dior, Tom Ford).	Should
4	Create Product	POST	/products	Admin adds perfume product including brand, description, price, and stock.	Must
5	List / Get Products	GET	/products , /products/:id	Retrieve list of perfumes or details of one perfume.	Must
6	Filter Products	GET	/products?brand=&price=	Customers can filter perfumes by brand or price range.	Should
7	Update Product	PATCH	/products/:id	Admin updates perfume price, stock, or description.	Must
8	Delete Product	DELETE	/products/:id	Admin removes product from catalog.	Should
9	Place Order	POST	/orders	Customer places order with product and quantity.	Must
10	View My Orders	GET	/orders/my-orders	Customer retrieves their order history.	Must
11	Admin View All Orders	GET	/orders	Admin retrieves all orders in the system.	Must
12	Update Order Status	PATCH	/orders/:id/status	Admin updates order status (pending, shipped, delivered).	Must

3. Database Entities & Relationships (Mandatory)

The system must demonstrate multiple relationship types for realistic database design.

Entity	Key Fields	Relationships Implemented	Type
User	id, fullName, email, password, role	One User → Many Orders	One-to-Many
Brand	id, name, description	One Brand → Many Products	One-to-Many
Product	id, name, brandId, description, price, stockQuantity	One Product → Many OrderItems	One-to-Many
Order	id, userId, status, totalPrice, createdAt	One Order → Many OrderItems	One-to-Many
OrderItem	id, orderId, productId, quantity, price	Many OrderItems ← One Product & One Order	Many-to-One

Required relationship types demonstrated:

• One-to-Many / Many-to-One (User → Orders)

• One-to-Many (Brand → Products)

• Join entity (OrderItem) connecting Orders and Products

4. Email Notification (Mailer) – Mandatory Feature

Action	Trigger Timing	Recipient	Suggested Subject	Content Suggestion
Order placed	After successful order	Customer	Order Confirmation – Perfumed Elegance	Order summary, product names, quantity, thank-you message
Order shipped	When admin updates order status	Customer	Your Order Has Been Shipped	Notification including tracking or delivery message
Order delivered	After order marked delivered	Customer	Order Delivered	Confirmation message and appreciation note

5. Technical Requirements

Category	Requirement / Recommendation	Mandatory?
Language	TypeScript	Yes
Framework	NestJS (latest stable)	Yes
Authentication	JWT + @nestjs/passport + Role Guards	Yes
Database & ORM	PostgreSQL + TypeORM	Yes
Relations	Use @OneToMany / @ManyToOne decorators	Yes
Validation	class-validator + global ValidationPipe	Yes
Documentation	@nestjs/swagger – Swagger UI	Yes
Email Integration	@nestjs-modules/mailer + Nodemailer	Yes
Password Security	bcryptjs hashing	Yes
Configuration	@nestjs/config with .env file	Yes
Project Structure	Modules: auth, users, brands, products, orders, mail	Yes
Version Control	Git + GitHub repository with README	Yes

6. Deliverables Checklist

• Public GitHub repository

• README.md including project overview, tech stack, setup instructions

• Database schema or ER diagram

• API endpoint documentation (Swagger)

• Working Swagger UI interface

• Screenshots of email notifications

• Optional short demo video